

AccountController.java

```
package com.abcbank.ekyc.controller;

import com.abcbank.ekyc.dto.AccountRequestDTO;
import com.abcbank.ekyc.dto.AccountResponseDTO;
import com.abcbank.ekyc.service.AccountService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

/**
 * Controller cung cấp API quản lý thông tin tài khoản ngân hàng.
 */
@RestController
@RequestMapping("/api/v1/accounts")
@RequiredArgsConstructor
@Slf4j
public class AccountController {

    private final AccountService accountService;

    /**
     * API thực hiện đăng ký mở tài khoản ngân hàng mới qua luồng eKYC.
     * Kích hoạt validation dữ liệu đầu vào bằng cách sử dụng
     * annotation @Valid.
     *
     * @param request DTO chứa thông tin đăng ký của khách hàng.
     * @return ResponseEntity chứa DTO kết quả tài khoản đã đăng ký
     * thành công và mã trạng thái HTTP 201 (Created).
     */
    @PostMapping("/register")
    public ResponseEntity<AccountResponseDTO> registerAccount(@Valid
@RequestBody AccountRequestDTO request) {
        // [INFO] Ghi log khi bắt đầu nhận request đăng ký mới kèm
        thông tin Email/SĐT che giấu một phần (Masking)
        log.info("Bắt đầu nhận request đăng ký mở tài khoản mới qua API
        (Email: {}, SĐT: {})",
```

```

        maskEmail(request.getEmail()),
maskPhone(request.getPhone()));

        // Gọi service xử lý logic nghiệp vụ đăng ký mở tài khoản
        AccountResponseDTO response = accountService.register(request);

        // [INFO] Ghi log khi đăng ký tài khoản thành công
        log.info("Đăng ký tài khoản thành công cho khách hàng qua API.
Số tài khoản sinh ra: {}",
                response.getAccountNumber());

        // Trả về mã trạng thái HTTP 201 Created cùng với kết quả đăng
ký
        return new ResponseEntity<>(response, HttpStatus.CREATED);
    }

    /**
     * Che giấu một phần số điện thoại khách hàng để đảm bảo bảo mật
thông tin (PII).
     *
     * @param phone Số điện thoại gốc.
     * @return Số điện thoại đã được che giấu phần giữa.
     */
    private String maskPhone(String phone) {
        if (phone == null || phone.length() < 6) return "*****";
        return phone.substring(0, 3) + "*****" +
phone.substring(phone.length() - 3);
    }

    /**
     * Che giấu một phần địa chỉ email khách hàng để đảm bảo bảo mật
thông tin (PII).
     *
     * @param email Địa chỉ email gốc.
     * @return Email đã được che giấu phần tên tài khoản ở giữa.
     */
    private String maskEmail(String email) {
        if (email == null || !email.contains("@")) return "*****";
        String[] parts = email.split("@");
        String name = parts[0];
        String domain = parts[1];
        if (name.length() <= 2) {
            return name.charAt(0) + "***@" + domain;
        }
    }

```

```

        return name.substring(0, 2) + "****" +
name.substring(name.length() - 2) + "@" + domain;
    }
}

```

AccountServiceImpl.java

```

package com.abcbank.ekyc.service.impl;

import com.abcbank.ekyc.dto.AccountRequestDTO;
import com.abcbank.ekyc.dto.AccountResponseDTO;
import com.abcbank.ekyc.entity.Account;
import com.abcbank.ekyc.entity.AccountStatus;
import com.abcbank.ekyc.exception.DuplicateResourceException;
import com.abcbank.ekyc.repository.AccountRepository;
import com.abcbank.ekyc.service.AccountService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.Random;

/**
 * Lớp triển khai thực thể của AccountService, thực hiện các quy trình
 nghiệp vụ liên quan đến tài khoản.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class AccountServiceImpl implements AccountService {

    private final AccountRepository accountRepository;
    private final Random random = new Random();

    /**
     * Thực hiện đăng ký mở tài khoản ngân hàng mới qua luồng eKYC.
     * Quy trình bao gồm: kiểm tra trùng lặp CCCD và Email, sinh số tài
 khoản ngẫu nhiên không trùng lặp,
     * khởi tạo thực thể tài khoản với trạng thái PENDING và lưu vào cơ
 sở dữ liệu.
     *
     * @param request Thông tin yêu cầu đăng ký mở tài khoản.

```

```

        * @return Thông tin tài khoản sau khi đăng ký thành công.
        * @throws DuplicateResourceException nếu số CCCD hoặc Email đã
được đăng ký trước đó.
    */
    @Override
    @Transactional
    public AccountResponseDTO register(AccountRequestDTO request) {
        // [INFO] Ghi log khi bắt đầu nhận request xử lý nghiệp vụ kèm
thông tin Email/SĐT che giấu một phần
        log.info("Bắt đầu xử lý đăng ký tài khoản eKYC tại Service
(Email: {}, SĐT: {})",
            maskEmail(request.getEmail()),
maskPhone(request.getPhone()));

        // 1. Kiểm tra xem số CCCD đã tồn tại trong hệ thống chưa
        if
(accountRepository.existsByCitizenId(request.getCitizenId())) {
            // [ERROR] Ghi log lỗi khi CCCD bị trùng lặp
            log.error("Đăng ký thất bại: Số CCCD {} đã tồn tại trên hệ
thống", maskCitizenId(request.getCitizenId()));
            throw new DuplicateResourceException("Số CCCD đã tồn tại
trong hệ thống");
        }

        // 1.1. Kiểm tra xem địa chỉ Email đã tồn tại trong hệ thống
chưa
        if (accountRepository.existsByEmail(request.getEmail())) {
            // [ERROR] Ghi log lỗi khi Email bị trùng lặp
            log.error("Đăng ký thất bại: Email {} đã tồn tại trên hệ
thống", maskEmail(request.getEmail()));
            throw new DuplicateResourceException("Địa chỉ Email đã tồn
tại trong hệ thống");
        }

        // 2. Sinh số tài khoản ngân hàng tự động (đảm bảo tính duy
nhất)
        String accountNumber = generateUniqueAccountNumber();

        // [INFO] Ghi log khi sinh số tài khoản thành công
        log.info("Đã sinh số tài khoản ngân hàng tự động thành công:
{}", accountNumber);

        // 3. Khởi tạo thực thể Account với trạng thái mặc định là
PENDING
        Account account = Account.builder()

```

```

        .fullName(request.getFullName())
        .phone(request.getPhone())
        .email(request.getEmail())
        .citizenId(request.getCitizenId())
        .accountNumber(accountNumber)
        .status(AccountStatus.PENDING) // Trạng thái mặc định
là PENDING

        .build();

// 4. Lưu thông tin tài khoản vào cơ sở dữ liệu
Account savedAccount = accountRepository.save(account);

// [INFO] Ghi log khi lưu thông tin tài khoản thành công
log.info("Lưu tài khoản thành công vào Database với ID: {}, Số
tài khoản: {}",
        savedAccount.getId(), savedAccount.getAccountNumber());

// 5. Chuyển đổi thực thể sang DTO để trả về cho Client
return AccountResponseDTO.builder()
        .accountId(savedAccount.getId())
        .accountNumber(savedAccount.getAccountNumber())
        .status(savedAccount.getStatus())
        .build();
}

/**
 * Sinh số tài khoản ngân hàng ngẫu nhiên gồm 10 chữ số với tiền tố
mặc định của ABC Bank là "190"
 * và kiểm tra trùng lặp trong cơ sở dữ liệu để đảm bảo tính duy
nhất.
 *
 * @return Số tài khoản ngân hàng duy nhất.
 */
private String generateUniqueAccountNumber() {
    String prefix = "190"; // Tiền tố số tài khoản của ABC Bank
    String accountNumber;
    boolean isDuplicate;

    do {
        // Sinh ngẫu nhiên thêm 7 chữ số phía sau để tạo thành
chuỗi 10 chữ số
        StringBuilder sb = new StringBuilder(prefix);
        for (int i = 0; i < 7; i++) {
            sb.append(random.nextInt(10));
        }
    }

```

```

        accountNumber = sb.toString();

        // Kiểm tra xem số tài khoản này đã có trong cơ sở dữ liệu
chưa
        isDuplicate =
accountRepository.existsByAccountNumber(accountNumber);
        } while (isDuplicate); // Nếu trùng thì lặp lại để sinh số mới

        return accountNumber;
    }

    /**
     * Che giấu một phần số điện thoại khách hàng để đảm bảo bảo mật
thông tin (PII).
     */
    private String maskPhone(String phone) {
        if (phone == null || phone.length() < 6) return "*****";
        return phone.substring(0, 3) + "*****" +
phone.substring(phone.length() - 3);
    }

    /**
     * Che giấu một phần địa chỉ email khách hàng để đảm bảo bảo mật
thông tin (PII).
     */
    private String maskEmail(String email) {
        if (email == null || !email.contains("@")) return "*****";
        String[] parts = email.split("@");
        String name = parts[0];
        String domain = parts[1];
        if (name.length() <= 2) {
            return name.charAt(0) + "***@" + domain;
        }
        return name.substring(0, 2) + "*****" +
name.substring(name.length() - 2) + "@" + domain;
    }

    /**
     * Che giấu một phần số CCCD khách hàng để đảm bảo bảo mật thông
tin (PII).
     */
    private String maskCitizenId(String citizenId) {
        if (citizenId == null || citizenId.length() < 6) return
"*****";

```

```
        return citizenId.substring(0, 3) + "*****" +  
citizenId.substring(citizenId.length() - 3);  
    }  
}
```